

Practice 05 — AWS Storage (EBS and S3)

Objective: attach a block volume that survives reboots and can be restored from a snapshot, and set up a bucket that protects its objects from overwrites and public exposure. These are the skills from today's lecture.

Timebox: ~75 min of actual work. If you've been stuck on one step for more than 15 minutes, stop and ask. Don't push through alone. If class ends before you're done, submit what you have: partial work beats nothing.

Two tracks, same skills. Do **Track A** if you have AWS access (Learner Lab, free-tier or credits). Do **Track B** if you don't: it uses Linux loop devices for EBS and a local S3-compatible server for S3. Both tracks are graded the same way. Pick one; you don't need both.

Definition of done

- ☐ A new volume is formatted, mounted at a data directory, and listed in `/etc/fstab` by **UUID** (or image path in Track B) with `nofail`
- ☐ `sudo mount -a` succeeds and the mount survives a reboot (Track A) or a `umount + mount -a` (Track B)
- ☐ A snapshot was taken, a file was "lost", and it was recovered from the snapshot
- ☐ A bucket has **versioning enabled**; an overwritten object was restored to its previous version
- ☐ A **lifecycle rule** is applied and shown with `get-bucket-lifecycle-configuration`
- ☐ **Block Public Access** is on (all four settings `true`)
- ☐ A **presigned URL** downloads the object with plain `curl`
- ☐ Cleanup done (see the end of your track)
- ☐ `REPORT.md` (or a PDF) submitted with your commands, outputs or screenshots, and answers to the questions below

AWS → Linux mapping

AWS	Track B (Linux / local)
EBS volume	Sparse image file + loop device (<code>truncate</code> , <code>losetup</code>)
Attach volume	<code>losetup --find --show <file></code>
<code>/dev/nvme1n1</code>	<code>/dev/loopN</code>
EBS snapshot	Copy of the unmounted image file / LVM snapshot (stretch)
Restore volume from snapshot	Mount the snapshot image as a new loop device
S3 bucket / objects	Local moto server (S3-compatible API) + <code>aws --endpoint-url</code>
S3 versioning, lifecycle, BPA, presign	Same <code>aws s3api</code> commands, pointed at the local server

Track A — real AWS

You need a running **EC2 instance** (Amazon Linux 2023, from week 3) that you can SSH into, plus the AWS CLI configured on your laptop or in CloudShell. Pick one region and stay in it.

A1. Create a volume in the instance's AZ

```
IID=i-0123456789abcdef0      # your instance ID
AZ=$(aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].Placement.AvailabilityZone' --output text)
echo $AZ

VOL=$(aws ec2 create-volume --availability-zone $AZ --size 1 \
  --volume-type gp3 --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=lab05-data}]' \
  --query VolumeId --output text)
aws ec2 wait volume-available --volume-ids $VOL

aws ec2 attach-volume --volume-id $VOL --instance-id $IID --device /dev/sdf
```

(Console alternative: EC2 → Volumes → Create volume, **same AZ as the instance**, gp3, 1 GiB, encrypted → Actions → Attach.)

A2. Format, mount, persist (on the instance)

```
lsblk                                # find the new 1G disk, e.g. nvme1n1
sudo file -s /dev/nvme1n1            # must print "data" (empty). If not, STOP.
sudo mkfs -t xfs /dev/nvme1n1
sudo mkdir -p /data
sudo mount /dev/nvme1n1 /data
echo "important data $(date)" | sudo tee /data/important.txt

sudo blkid /dev/nvme1n1              # copy the UUID
sudo cp /etc/fstab /etc/fstab.bak
echo 'UUID=<your-uuid> /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab

sudo umount /data && sudo mount -a    # no errors = fstab line is valid
findmnt /data
sudo reboot                          # reconnect, then:
findmnt /data && cat /data/important.txt
```

A3. Snapshot → lose a file → restore

```
# laptop / CloudShell
SNAP=$(aws ec2 create-snapshot --volume-id $VOL --description "lab05 before disaster" \
  --query SnapshotId --output text)
aws ec2 wait snapshot-completed --snapshot-ids $SNAP
```

```
# instance – the "disaster"
sudo rm /data/important.txt
```

```
# laptop / CloudShell – new volume from the snapshot, same AZ
VOL2=$(aws ec2 create-volume --availability-zone $AZ --snapshot-id $SNAP \
  --volume-type gp3 --query VolumeId --output text)
aws ec2 wait volume-available --volume-ids $VOL2
aws ec2 attach-volume --volume-id $VOL2 --instance-id $IID --device /dev/sdg
```

```
# instance
lsblk                                # e.g. nvme2n1
sudo file -s /dev/nvme2n1            # shows XFS → do NOT mkfs
sudo mkdir -p /restore
sudo mount -o ro,nouuid /dev/nvme2n1 /restore
sudo cp /restore/important.txt /data/
cat /data/important.txt
```

`nouuid` is needed because the restored XFS filesystem has the **same UUID** as the one already mounted at `/data`.

A4. S3 bucket with versioning

Bucket names are global, so make yours unique (e.g. add your student ID).

```
B=css4005-lab05-<studentid>
REGION=$(aws configure get region)

# us-east-1: omit --create-bucket-configuration entirely
aws s3api create-bucket --bucket $B --create-bucket-configuration LocationConstraint=$REGION

aws s3api put-bucket-versioning --bucket $B --versioning-configuration Status=Enabled
aws s3api get-bucket-versioning --bucket $B

echo "version 1" > notes.txt;      aws s3 cp notes.txt s3://$B/notes.txt
echo "version 2 (oops)" > notes.txt; aws s3 cp notes.txt s3://$B/notes.txt

aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[].{Id:VersionId,Latest:IsLatest,Size:Size}' --output table

OLD=$(aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[?IsLatest==`false`].VersionId' --output text)
aws s3api get-object --bucket $B --key notes.txt --version-id "$OLD" restored.txt
aws s3 cp restored.txt s3://$B/notes.txt      # old content is the latest again
aws s3 cp s3://$B/notes.txt -                  # → version 1
```

Then delete the object and bring it back:

```
aws s3 rm s3://$B/notes.txt
aws s3api list-object-versions --bucket $B --prefix notes.txt # see the DeleteMarkers
DM=$(aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'DeleteMarkers[?IsLatest==`true`].VersionId' --output text)
aws s3api delete-object --bucket $B --key notes.txt --version-id "$DM"
aws s3 cp s3://$B/notes.txt -                  # it's back
```

A5. Lifecycle, Block Public Access, presigned URL

Save as `lifecycle.json`:

```
{
  "Rules": [
    {
      "ID": "logs-to-ia-then-expire",
      "Filter": { "Prefix": "logs/" },
      "Status": "Enabled",
      "Transitions": [ { "Days": 30, "StorageClass": "STANDARD_IA" } ],
      "Expiration": { "Days": 365 },
      "NoncurrentVersionExpiration": { "NoncurrentDays": 30 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket $B --lifecycle-configuration
file:///lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket $B

aws s3api put-public-access-block --bucket $B --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api get-public-access-block --bucket $B

curl -s -o /dev/null -w "%{http_code}\n" https://$B.s3.$REGION.amazonaws.com/notes.txt # 403 -
private
URL=$(aws s3 presign s3://$B/notes.txt --expires-in 300)
curl -s "$URL" # works
```

A6. Cleanup (do not skip, because forgotten resources cost money)

```
# instance: unmount and restore the original fstab
sudo umount /restore /data
sudo cp /etc/fstab.bak /etc/fstab

# laptop / CloudShell
aws ec2 detach-volume --volume-id $VOL2; aws ec2 detach-volume --volume-id $VOL
aws ec2 wait volume-available --volume-ids $VOL $VOL2
aws ec2 delete-volume --volume-id $VOL; aws ec2 delete-volume --volume-id $VOL2
aws ec2 delete-snapshot --snapshot-id $SNAP

# a versioned bucket must be emptied of ALL versions and delete markers first
aws s3api list-object-versions --bucket $B --output json --query \
  '{Objects: [Versions[].{Key:Key,VersionId:VersionId}, DeleteMarkers[.
{Key:Key,VersionId:VersionId}]]}' > del.json
aws s3api delete-objects --bucket $B --delete file:///del.json
aws s3api delete-bucket --bucket $B
```

Stop or terminate the instance if you don't need it for next week.

Track B — no AWS (Linux + local S3)

Part 1 needs Linux with `sudo` : an Ubuntu VM, WSL2 on Windows, or any Linux machine. It will **not** work on macOS. Part 2 (S3) works on any OS with Python 3.

B1. Create and "attach" a volume

```
truncate -s 1G ~/ebs-vol1.img          # sparse 1 GiB file = the "EBS volume"
DEV=$(sudo losetup --find --show ~/ebs-vol1.img)
echo $DEV                               # e.g. /dev/loop3
lsblk $DEV
```

B2. Format, mount, persist

```
sudo file -s $DEV                      # "data" → empty, safe to format
sudo mkfs.ext4 -L lab05data $DEV
sudo mkdir -p /mnt/data
sudo mount $DEV /mnt/data
echo "important data $(date)" | sudo tee /mnt/data/important.txt
sudo blkid $DEV                        # note the UUID (report question 2)
```

A loop device doesn't exist at boot until something creates it, so `/etc/fstab` refers to the **image file** with the `loop` option (mount sets up the loop device itself):

```
sudo umount /mnt/data && sudo losetup -d $DEV
sudo cp /etc/fstab /etc/fstab.bak
echo "$HOME/ebs-vol1.img /mnt/data ext4 loop,nofail 0 0" | sudo tee -a /etc/fstab
sudo mount -a
findmnt /mnt/data && cat /mnt/data/important.txt
```

B3. Snapshot → lose a file → restore

```
sudo umount /mnt/data # consistent copy: unmount first
cp --sparse=always ~/ebs-vol1.img ~/snap-1.img
sudo mount -a # back online

sudo rm /mnt/data/important.txt # the "disaster"

sudo mkdir -p /mnt/restore
sudo mount -o loop,ro ~/snap-1.img /mnt/restore
sudo cp /mnt/restore/important.txt /mnt/data/
cat /mnt/data/important.txt
```

This copy is a **full** copy, unlike EBS snapshots, which are incremental. The stretch task below shows a copy-on-write snapshot that is closer to how EBS works.

Stretch (optional): LVM snapshot

```
truncate -s 2G ~/lvm-disk.img
LDEV=$(sudo losetup --find --show ~/lvm-disk.img)
sudo pvcreate $LDEV && sudo vgcreate labvg $LDEV
sudo lvcreate -L 1G -n data labvg
sudo mkfs.ext4 /dev/labvg/data
sudo mkdir -p /mnt/lvdata && sudo mount /dev/labvg/data /mnt/lvdata
echo "hello" | sudo tee /mnt/lvdata/file.txt

sudo lvcreate -s -L 200M -n data-snap /dev/labvg/data # stores only changed blocks
sudo rm /mnt/lvdata/file.txt
sudo mkdir -p /mnt/lvsnap && sudo mount -o ro /dev/labvg/data-snap /mnt/lvsnap
cat /mnt/lvsnap/file.txt # still there in the snapshot
sudo lvs labvg # Data% = how much changed

# cleanup
sudo umount /mnt/lvsnap /mnt/lvdata
sudo lvremove -y labvg && sudo vgremove labvg && sudo pvremove $LDEV
sudo losetup -d $LDEV && rm ~/lvm-disk.img
```

B4. Start a local S3-compatible server

[moto](#) emulates the S3 API locally. Data lives in memory and disappears when you stop the server.

```
python3 -m venv ~/lab05-venv && source ~/lab05-venv/bin/activate
pip install 'moto[server]' awscli
moto_server -p 5001 & # leave it running

export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=eu-central-1
alias s3l='aws --endpoint-url http://127.0.0.1:5001'
```

Every command below is the same as in Track A, with `s3l` instead of `aws`.

B5. Bucket, versioning, restore

```
B=css4005-lab05-<studentid>
s3l s3api create-bucket --bucket $B --create-bucket-configuration LocationConstraint=eu-central-1
s3l s3api put-bucket-versioning --bucket $B --versioning-configuration Status=Enabled
s3l s3api get-bucket-versioning --bucket $B

echo "version 1" > notes.txt; s3l s3 cp notes.txt s3://$B/notes.txt
echo "version 2 (oops)" > notes.txt; s3l s3 cp notes.txt s3://$B/notes.txt
s3l s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[].{Id:VersionId,Latest:IsLatest,Size:Size}' --output table

OLD=$(s3l s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[?IsLatest==`false`].VersionId' --output text)
s3l s3api get-object --bucket $B --key notes.txt --version-id "$OLD" restored.txt
s3l s3 cp restored.txt s3://$B/notes.txt
s3l s3 cp s3://$B/notes.txt - # → version 1

s3l s3 rm s3://$B/notes.txt
DM=$(s3l s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'DeleteMarkers[?IsLatest==`true`].VersionId' --output text)
s3l s3api delete-object --bucket $B --key notes.txt --version-id "$DM"
s3l s3 cp s3://$B/notes.txt - # it's back
```

B6. Lifecycle, Block Public Access, presigned URL

Create `lifecycle.json` exactly as in **A5**, then:

```
s3l s3api put-bucket-lifecycle-configuration --bucket $B --lifecycle-configuration
file://lifecycle.json
s3l s3api get-bucket-lifecycle-configuration --bucket $B

s3l s3api put-public-access-block --bucket $B --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
s3l s3api get-public-access-block --bucket $B

URL=$(s3l s3 presign s3://$B/notes.txt --expires-in 300)
echo "$URL"; curl -s "$URL"
```

The local server **stores** the lifecycle and public-access settings but doesn't actually move objects between storage classes or block anyone. In your report, explain what real S3 would do with each setting.

B7. Cleanup

```
sudo umount /mnt/restore /mnt/data
sudo cp /etc/fstab.bak /etc/fstab
rm ~/ebs-vol1.img ~/snap-1.img
pkill -f moto_server # stops the local S3 (all its data is gone)
deactivate
```

Report questions

Answer briefly (2–4 sentences each) in `REPORT.md` :

1. Why must the EBS volume be in the same AZ as the instance? How would you get the same data onto an instance in another AZ?
2. Why does `/etc/fstab` use the UUID instead of `/dev/nvme1n1` ? What does `nofail` protect you from?
3. EBS snapshots are *incremental*. What does that mean, and why does it matter for cost?
4. What happens to an object when you run `aws s3 rm` on a versioned bucket? How did you get it back?
5. Which storage class would you choose for: (a) user avatars, (b) logs needed for 1 year, rarely read, (c) 7-year compliance archives?
6. Why did the plain `https://...` URL to your object fail while the presigned URL worked? (Track B: explain what real S3 would return.)

Common mistakes

- **mkfs on the restored volume.** It already has your data; formatting it erases the restore. Always run `sudo file -s <device>` first.
- **"wrong fs type" / "duplicate UUID" mounting the restored XFS volume.** The snapshot copy has the same UUID as the mounted original; use `mount -o nouuid` (Track A).

- **fstab typo** → **instance won't boot**. Always run `sudo mount -a` after editing `fstab` and *before* rebooting, and keep `nofail`.
- **Can't attach the volume**. It's in a different AZ than the instance.
- **IllegalLocationConstraintException on create-bucket**. In `us-east-1`, omit `--create-bucket-configuration`; in every other region, include it with that region's name.
- **BucketNotEmpty on delete**. Versioned buckets keep old versions and delete markers; use the `delete-objects` cleanup in A6.
- **Forgot cleanup**. `available` volumes and old snapshots are billed even though nothing uses them.